

第3周 第1课时 现代计算系统结构与算力技术

单芯片集成与异构架构

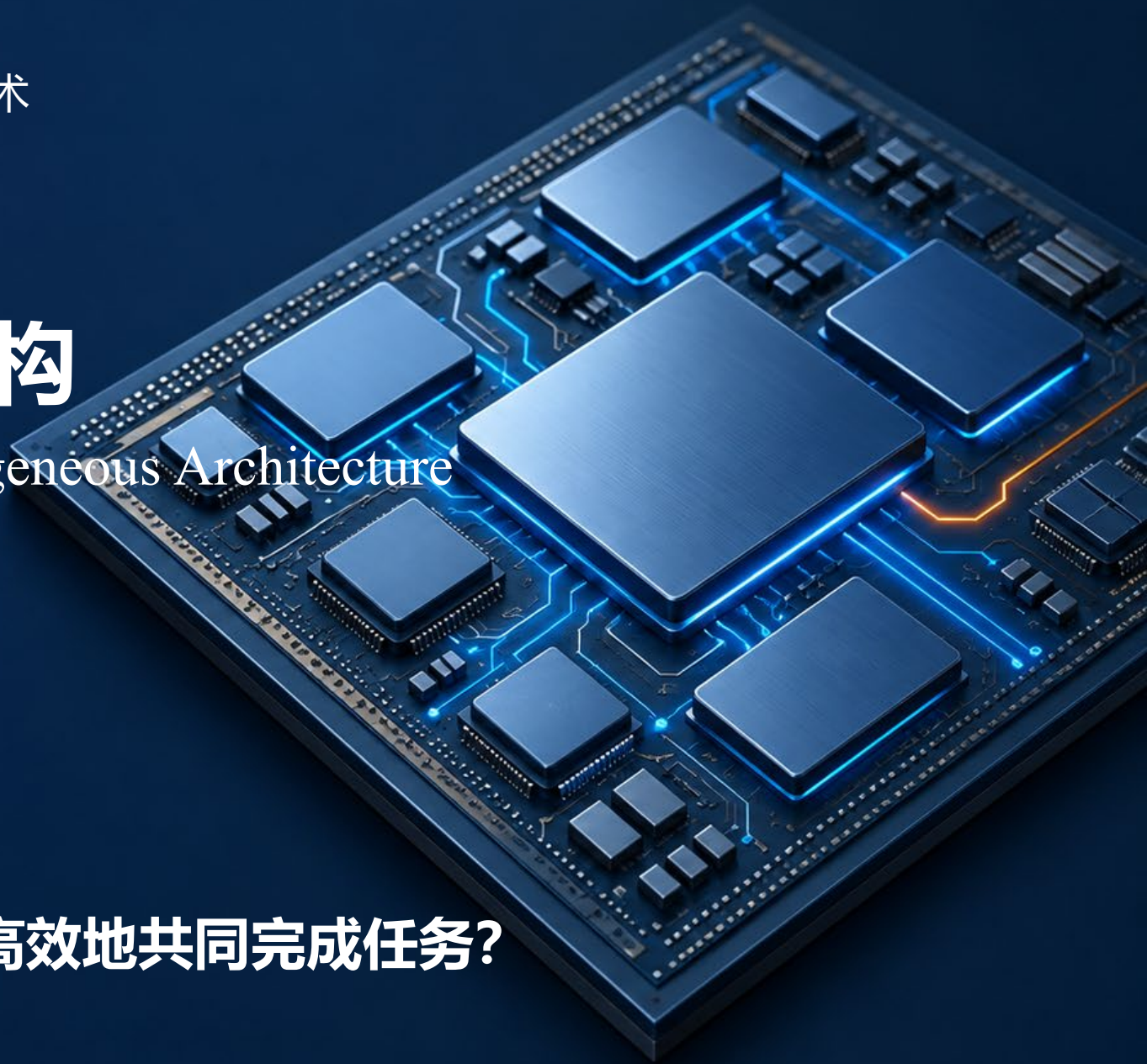
Single-chip Integration and Heterogeneous Architecture

主讲人：焦庆春、刘德康

自动化与电气工程学院

智能科学与技术系

核心问题：多个核心怎样正确、高效地共同完成任务？





AMD

像我一样
战斗!

选锐龙

618买电脑选锐龙

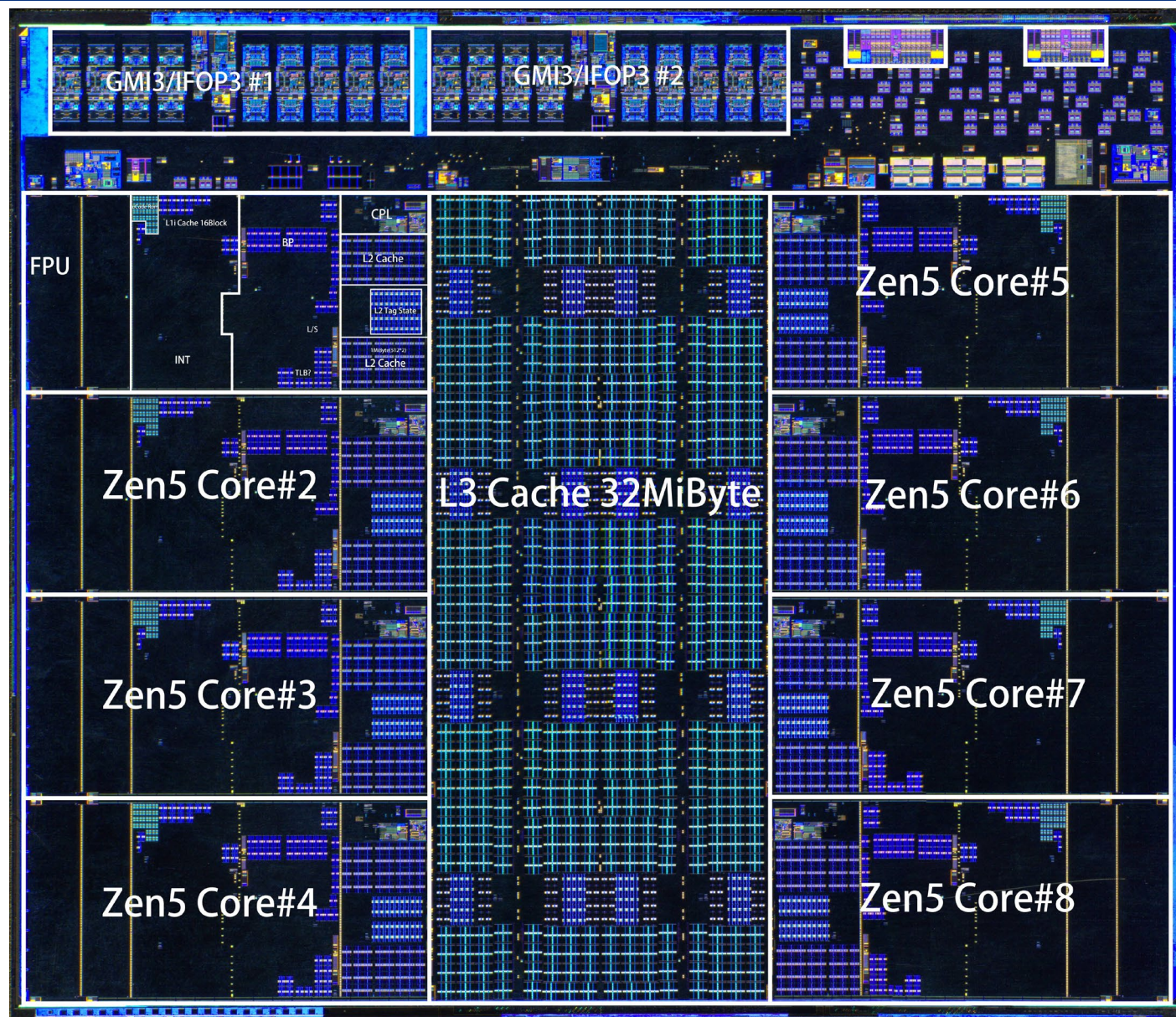
晒单返百元E卡*

时间: 5.23-6.24

AMD
RYZEN
6000 SERIES

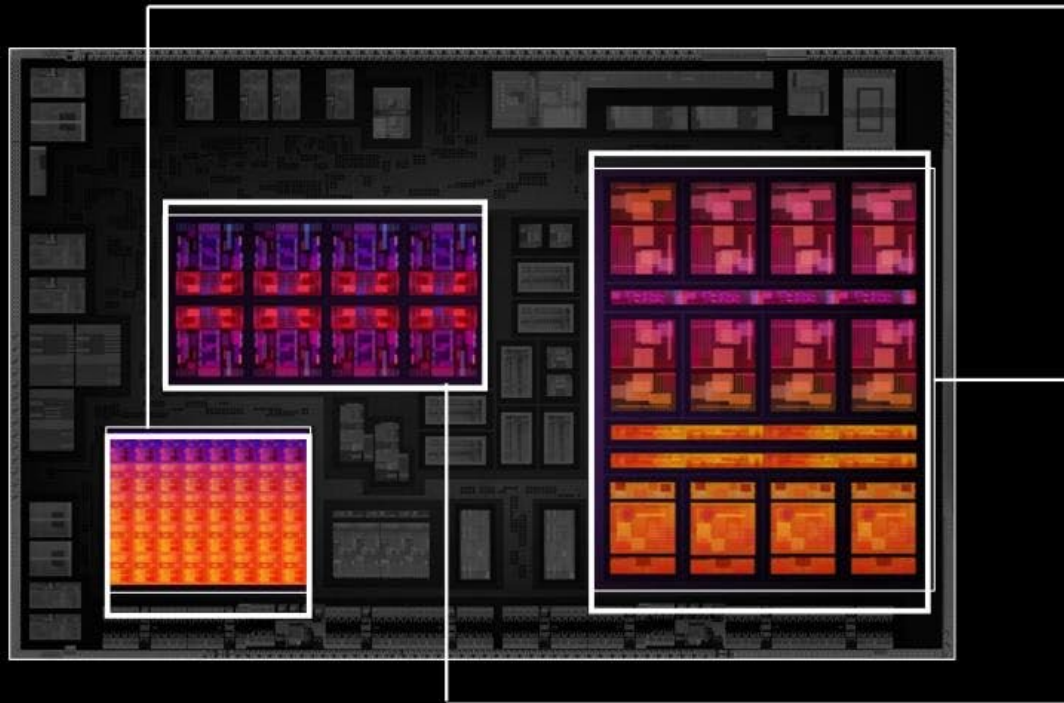
AMD锐龙芯鉴大使
UZI 简自豪

*活动规则以店铺活动页为准



3rd Gen AMD Ryzen™ AI

Next-gen AI PC experiences demand the best of NPU, CPU, and GPU architectures



AMD
XDNA 2

Next-Gen NPU
up to 50 TOPS



Next-Gen CPU
Up to 12 Cores, 24 Threads

AMD
RDNA 3.5

Next-Gen GPU
Up to 16 compute units

01 多核架构与线程级并行

讲义3.1 · P45 – P49

02 缓存一致性与MESI

讲义3.2.1 · P49 – P52

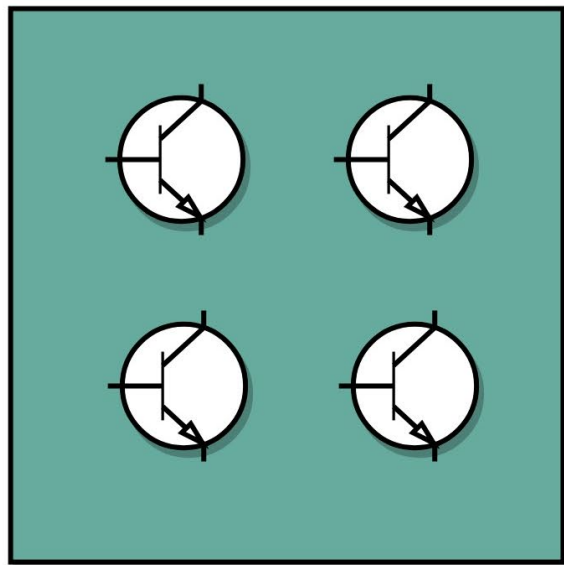
03 存储墙与多级缓存

讲义3.2.2 · P52 – P54

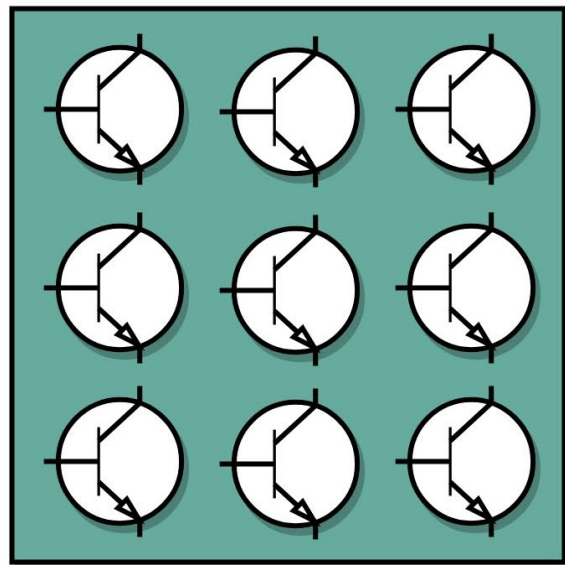
04 均匀访问与NUMA

讲义3.3.3 · P54 – P56

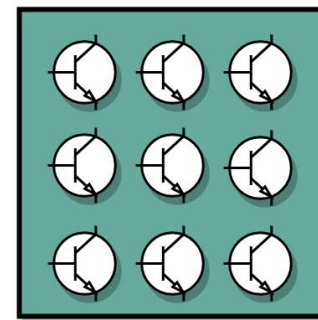
- CPU功耗通常分为**静态功耗**和**动态功耗**两部分
- 静态功耗：与工艺、温度和电压相关，主要来源于晶体管泄露电流，无法精确计算
- 动态功耗：与CPU的**工作频率**、**核心数量**和**电压**有关



芯片内部的晶体管

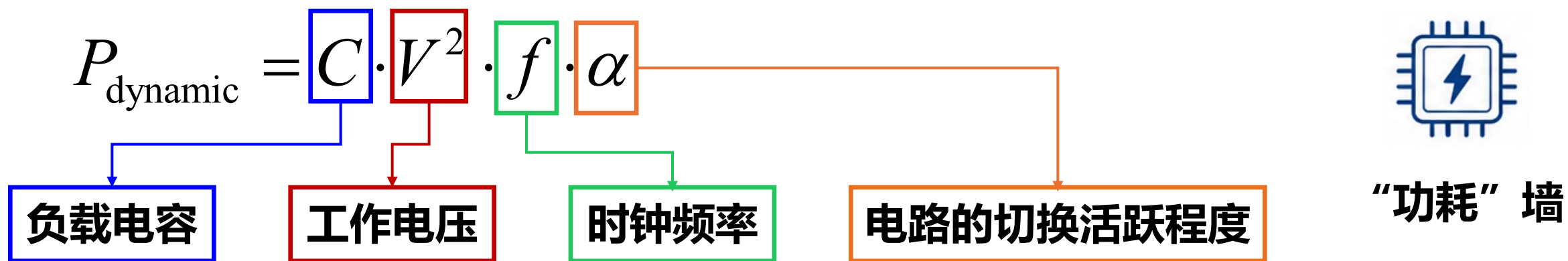


更多数量晶体管带来
能耗和散热挑战



制程的提升解决了能耗
和散热问题，还带来更
小的芯片尺寸

- CPU功耗通常分为**静态功耗**和**动态功耗**两部分
- 动态功耗：与CPU的**工作频率**、**核心数量**和**电压**有关
- 提高频率带来的功耗增长，常常超过性能增长。关键在于，**升频往往还需要升压**

$$P_{\text{dynamic}} = C \cdot V^2 \cdot f \cdot \alpha$$


负载电容

工作电压

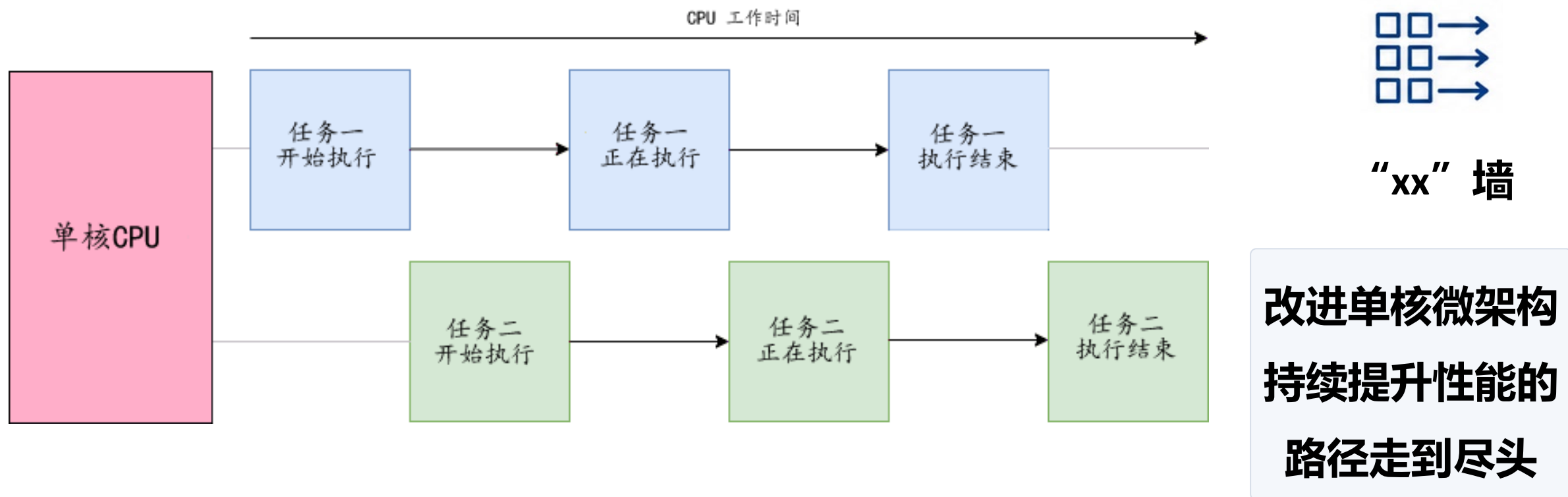
时钟频率

电路的切换活跃程度

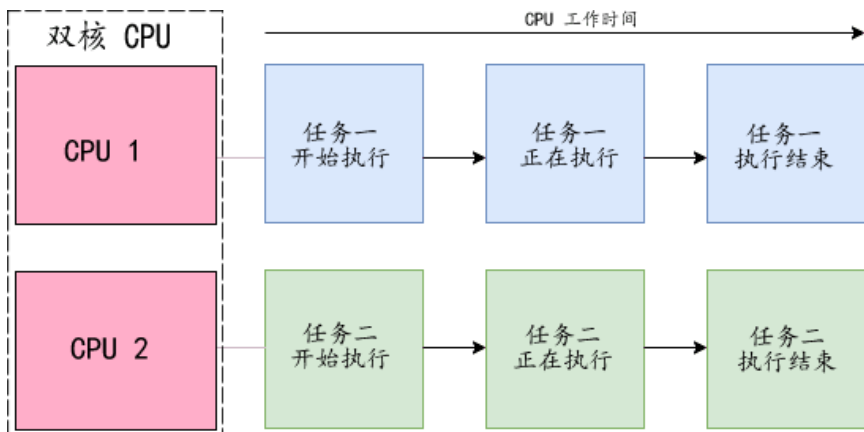
“功耗”墙

情况	假设	动态功耗变化
只提高频率	电压不变，频率提高20%	增加约20%
升频同时升压	频率提高20%，电压提高10%	$1.2 \times 1.1^2 = 1.452$ ，增加约45.2%

- 超标量架构、乱序执行技术、分支预测技术等复杂的“微架构”技术
- 单核处理器在一个时钟周期内能够执行更多指令
- 更深流水线、更大重排序缓冲区等，榨取ILP的硬件复杂度和功耗成本指数级增长



01 从追求单核频率到追求多核能效



多核架构通过在**单个芯片上集成多个相对简单、能效更高的处理核心**，在**控制总功耗的前提下**，通过**线程级并行**来提升系统整体吞吐量

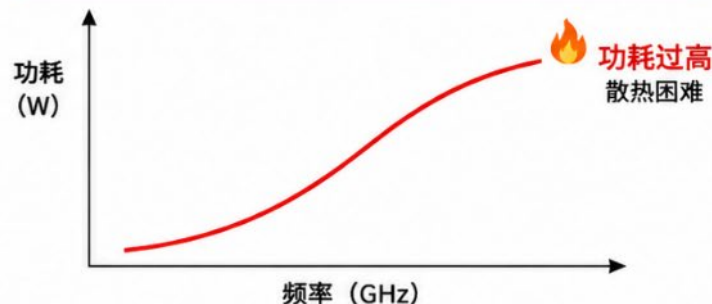
联想拯救者R9000P

- ❑ AMD锐龙9 8945HX处理器
- ❑ 16核心32线程
- ❑ 单核MAX主频（睿频）5.4GHz
- ❑ CPU主频 2.5GHz

单核性能扩展的瓶颈

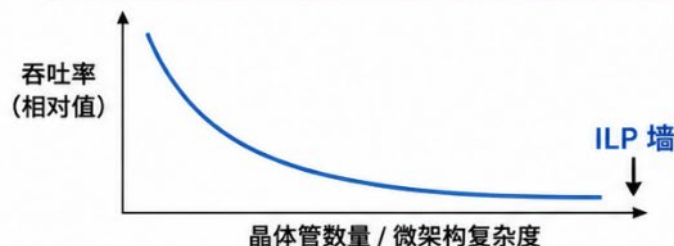
频率提升与指令级并行的限制

1. 频率提升受限，功耗急剧上升



频率越高，动态功耗与漏电功耗急剧增加，导致发热严重，**制约了单核性能的继续提升**。

2. 指令级并行 (ILP) 收益递减



随着硬件复杂度和晶体管数量的增加，可挖掘的指令级并行性越来越有限，**收益显著递减**。

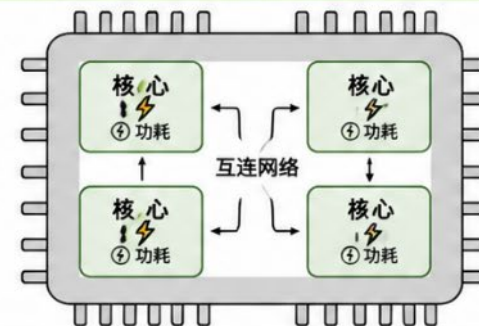


总结：单核性能受制于功耗墙和 ILP 墙，难以持续提升；多核架构通过并行扩展，在相同功耗下带来更高性能与能效，是现代处理器发展的主流方向。

多核架构的优势

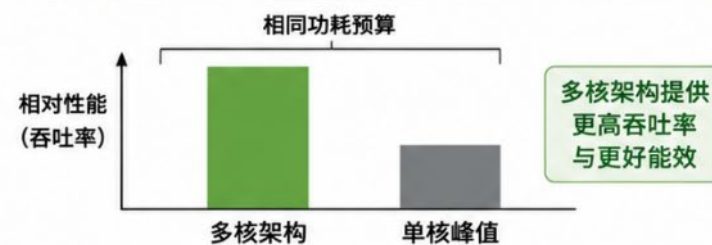
更高的能效与系统吞吐率

1. 多核：在相同功耗预算下提供更高性能



将芯片面积分配给多个核心，通过片上互连协同工作，在不增加整体功耗的前提下，**获得更高系统吞吐率**。

2. 相同功耗预算下的性能对比



多核架构在相同功耗预算下，能够实现**更高的吞吐率**和**更优的能效比**。

□ 01 并行对象；02 主要组织者；03 典型硬件支持；04 主要限制

指令级并行 ILP

01 同一线程中的指令

02 编译器与核心内调度逻辑

03 流水线、超标量、乱序执行

04 数据依赖、控制依赖

线程级并行 TLP

01 多个线程或任务

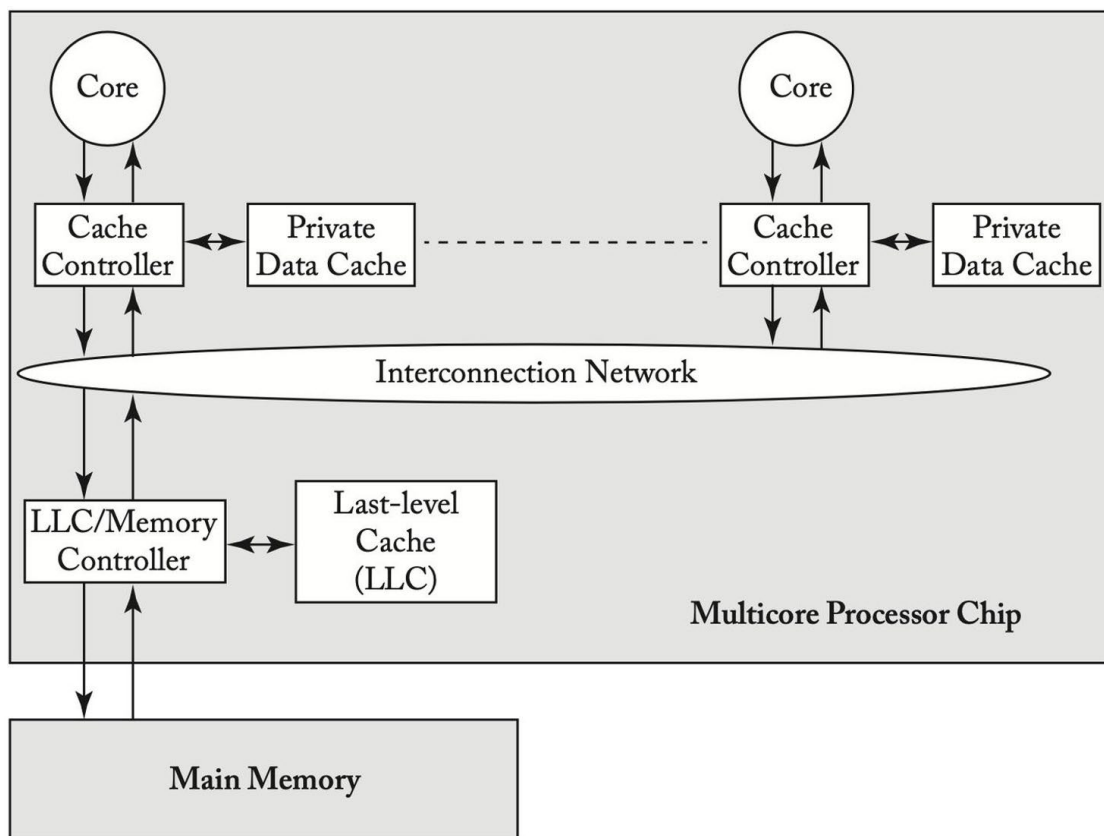
02 程序、运行时与操作系统

03 多个核心与共享资源

04 串行部分、同步和资源竞争

多个核心内部，依然可以继续利用 ILP（指令级并行）

□ 线程级并行 TLP：通过同时执行多个控制流（线程）来提升系统吞吐量



对称多处理架构 (SMP)

□ 核心对等：

操作系统调度，平等地执行所有任务

□ 缓存分层：

私有缓存与共享资源协作

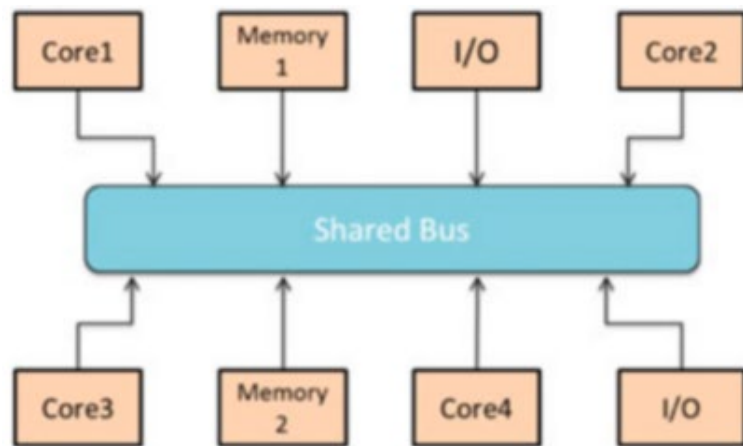
□ 内存共享：

多个核心访问同一地址空间

性能优势： 当一个任务被阻塞时，操作系统可以立即将另一个就绪任务调度到空闲核心上执行

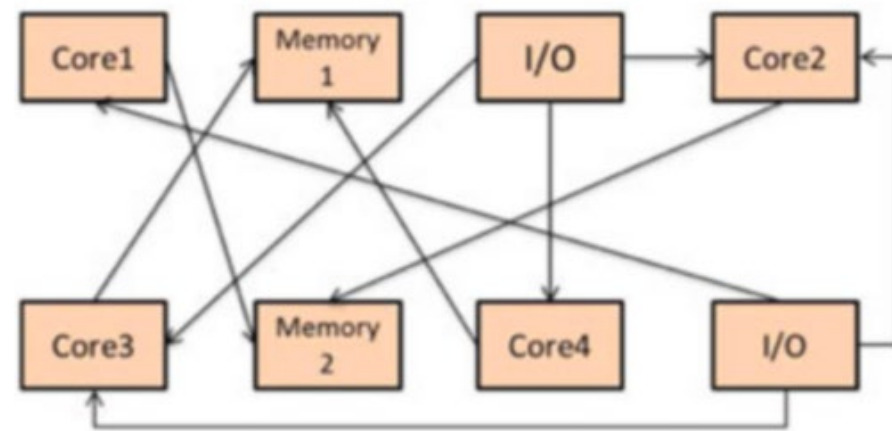
架构瓶颈： 对共享的系统总线来连接所有核心和内存的依赖导致的资源竞争，限制了SMP架构可扩展性

□ 线程级并行 TLP：通过同时执行多个控制流（线程）来提升系统吞吐量



共享总线

共享通信介质，容易形成争用



交叉开关

允许多路通信，增加交换与布线成本

互联拓扑的选择直接决定了在芯片上增加更多核心能否有效转化为更高的实际性能

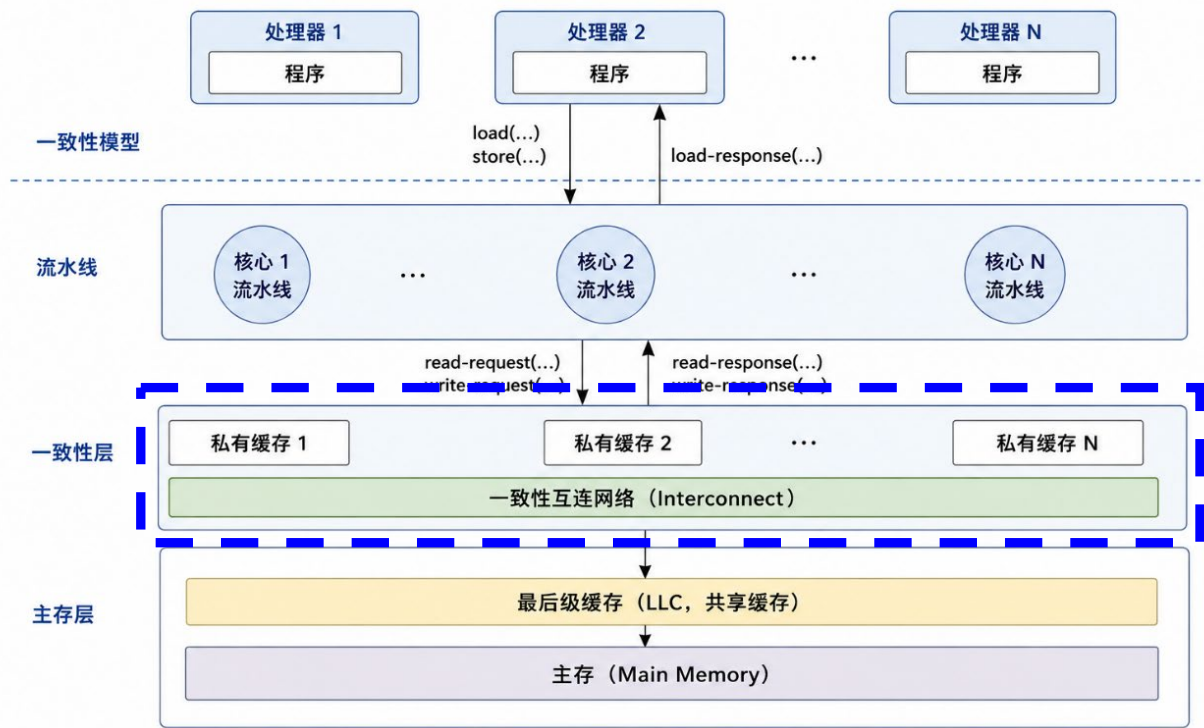
多线程编程

数据竞争 / 同步

负载均衡

可扩展性

- 多核处理器中，每个核心通常配备独立的高速缓存（Cache）以减少内存访问延迟
- 问题：同一内存地址的数据可能在多个核心的缓存中存在副本，当其中一个副本被修改时，如何保证其它副本的正确性？

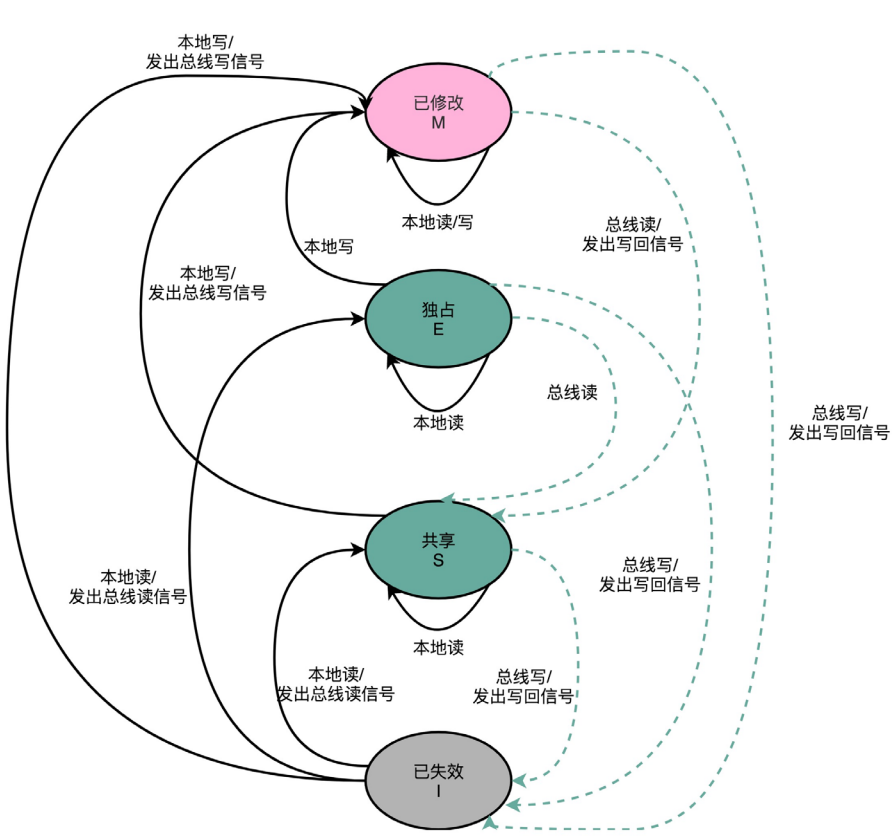


多处理器缓存一致性协议结构

缓存一致性协议：建立数据共享的规则

- 写传播 (Write Propagation):** 任意一个核心对某个数据的修改，能够被其它所有缓存了该数据副本的核心所看到
- 事务串行化 (Transaction Serialization):** 对所有核心而言，它们观察到的对同一内存地址的读写操作顺序应该是一致的
- 原子性 (Atomicity):** 对单个内存地址的读操作必须能够获取到最近一次对该地址的写操作所写入的值

- 总线侦听协议是依赖于一个所有核心都能访问的共享广播介质：系统总线
- 每个核心的缓存控制器都会持续监控总线上的所有事务
- MESI协议：通过明确的状态规则避免数据访问冲突，状态标记、总线嗅探

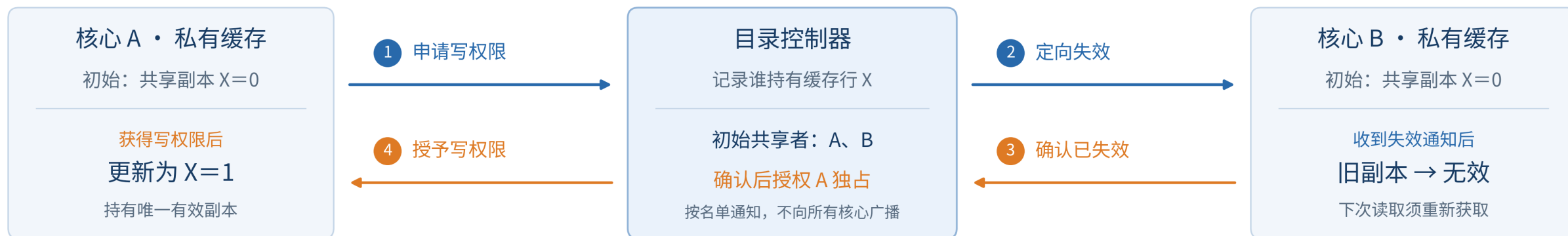


情况	状态标记 / 描述	总线嗅探 / 监听任务
M 修改	该缓存行有效，数据被修改了，和内存中的数据不一致，数据只存在于本缓存中	缓存行必须时刻监听所有试图读该缓存行相对旧主存的操作，这种操作必须在缓存将该缓存行写回主存并且将状态变为S之前被延迟执行
E 独享	该缓存行有效，数据和内存中的数据一致，数据只存在于本缓存中	缓存行必须监听其他缓存读该主存中该缓存行的操作，一旦有这种操作，该缓存行需要变成S状态
S 共享	该缓存行有效，数据和内存中的数据一致，数据存在于很多缓存中	缓存行必须监听其它缓存使该缓存行无效或者独享该缓存行的请求，并将该缓存行变成无效
I 无效	该缓存行无效	无

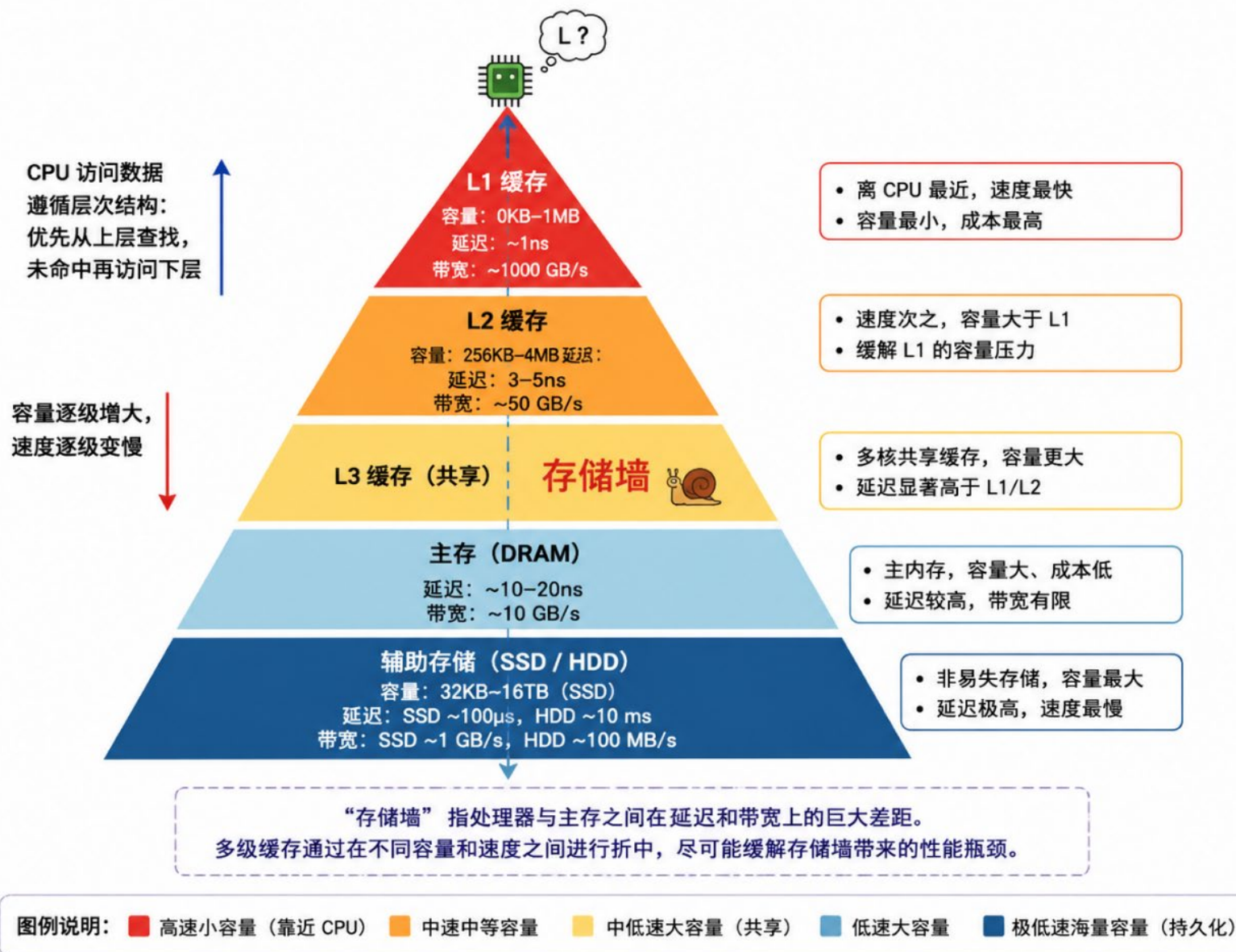
操作	核心 A	核心 B	主存 X
初始	I	I	0
A 读取 X	E, 值为 0	I	0
B 读取 X	S, 值为 0	S, 值为 0	0

操作完成后的状态	核心 A	核心 B	主存 X
两核共同读取后	S, 值为 0	S, 值为 0	0
A 取得写权限, 写 X = 1	M, 值为 1	I	0 (旧数据)
B 再次读取 X	S, 值为 1	S, 值为 1	1 (新数据)

- 总线侦听协议简单有效，但可扩展性受限于共享广播介质
- 目录协议摒弃广播机制，采用“精准定位”策略，采用“单播/组播”机制
- 计算机架构通过增加设计的复杂性来换取系统规模的拓展能力

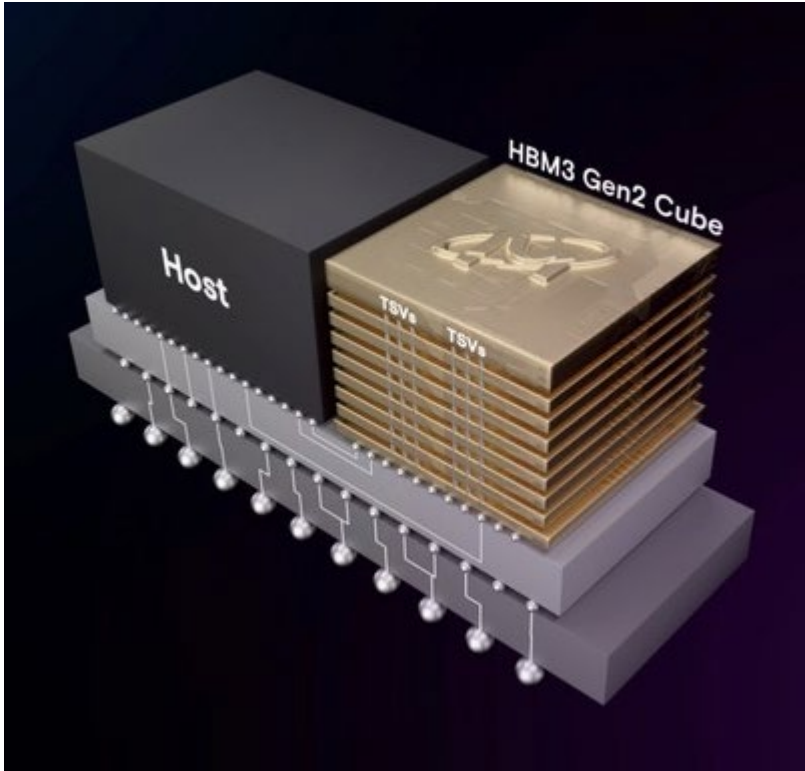


仅向那些缺失缓存了该数据副本的核心发送“无效化”或“更新”消息，而不是向所有核心广播

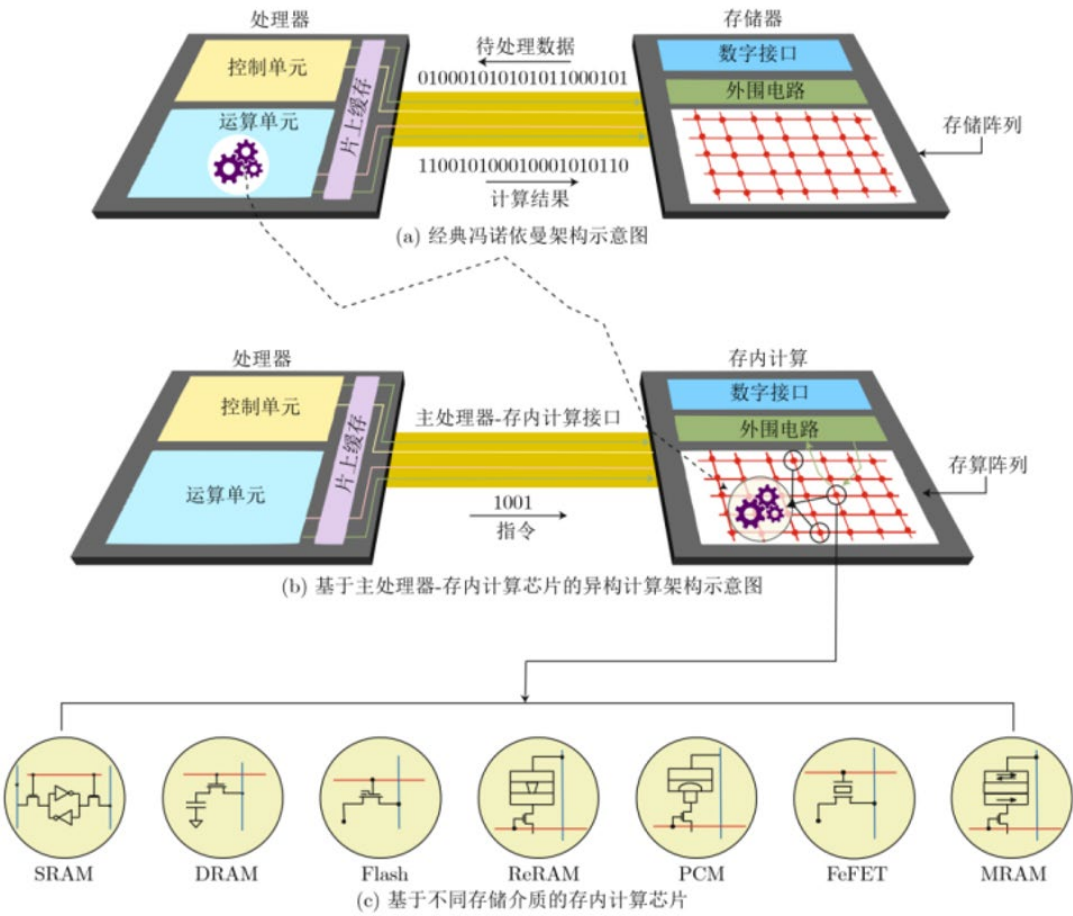


基于局部性原理的多级缓存体系

- 在快速的处理器寄存器与慢速的主存之间，插入**多级**速度与容量各不相同的**静态随机存取存储器**作为缓存
- 三级缓存：L1、L2、L3，容量依次增大，速度逐级变慢
- 当处理器需要数据时，首先在最快的L1缓存中寻找，若未找到（缓存缺失），则依次在L2、L3缓存中查找，最后才访问主存
- 通过利用程序访问的**时间局部性**和**空间局部性**，**缓存体系能够将大部分数据访问请求在快速缓存中得到满足**，从而将平均内存访问时间降低到可接受的水平，是缓解存储墙的主力军

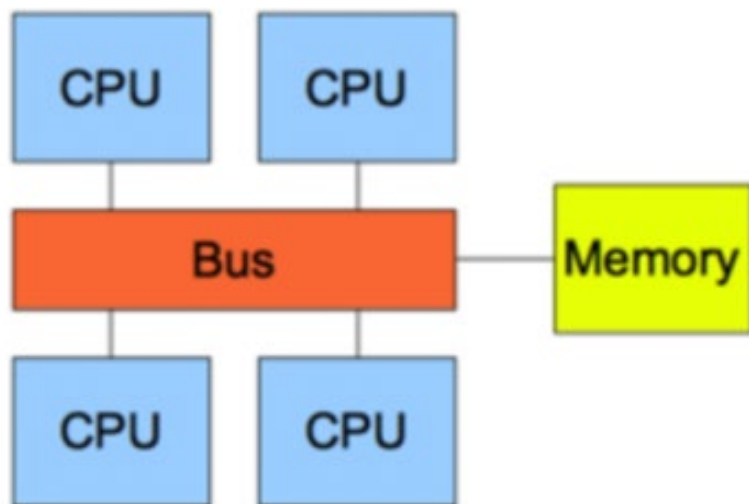


高带宽内存 HBM



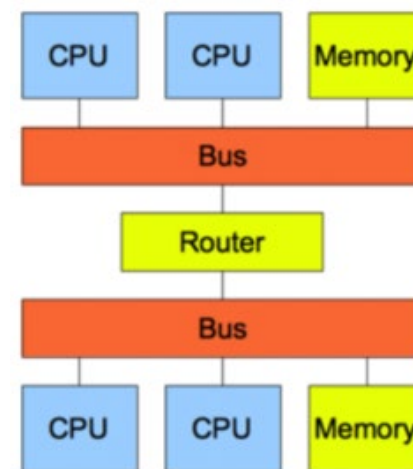
存内计算

□ 存储架构从简单的**对称多处理模型 SMP**，走向了**非均匀存储访问模型 NUMA**



SMP / UMA

- 统一内存访问（UMA）：对称性和对软件的透明度
- 任何核心访问任何内存地址的延迟都是相同的
- 共享总线是被所有核心争用的独占性资源
- 核心数量增加，总线仲裁开销急剧上升
- 传统SMP架构很难高效扩展至超过约8个核心的系统



NUMA

- NUMA架构将系统划分为多个相对独立的节点
- 每个节点包含一组处理器核心、本地内存和I/O资源
- 节点之间通过高速互联网连接
- 处理器核心访问本地节点内存的延迟显著低于访问远程节点内存的延迟

第3周 第2课时 现代计算系统结构与算力技术

单芯片集成与异构架构

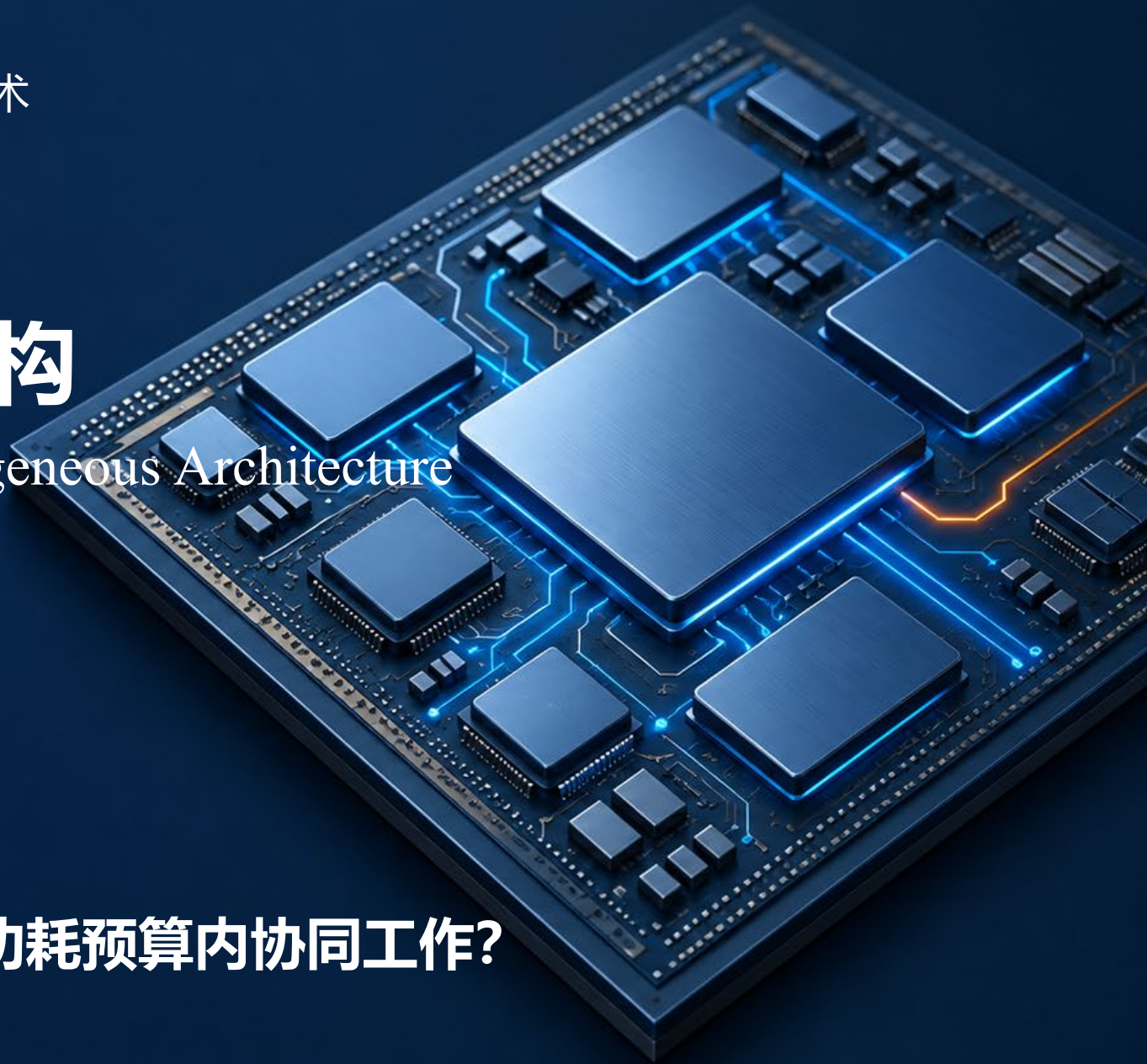
Single-chip Integration and Heterogeneous Architecture

主讲人：焦庆春、刘德康

自动化与电气工程学院

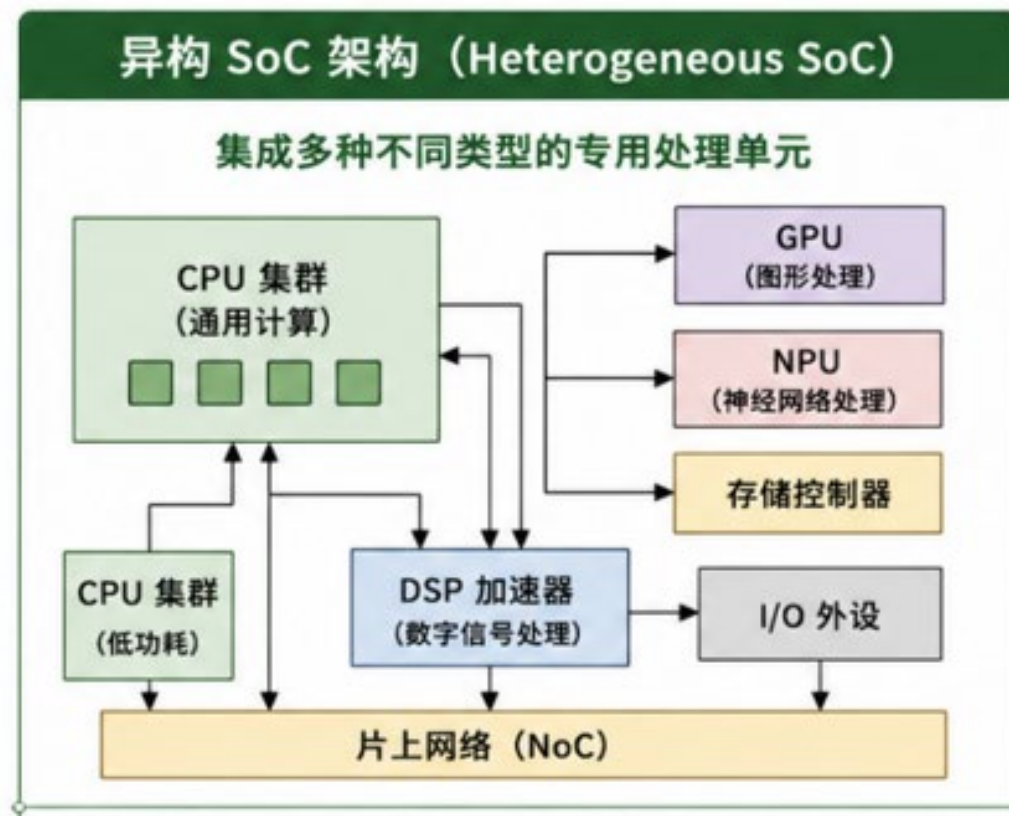
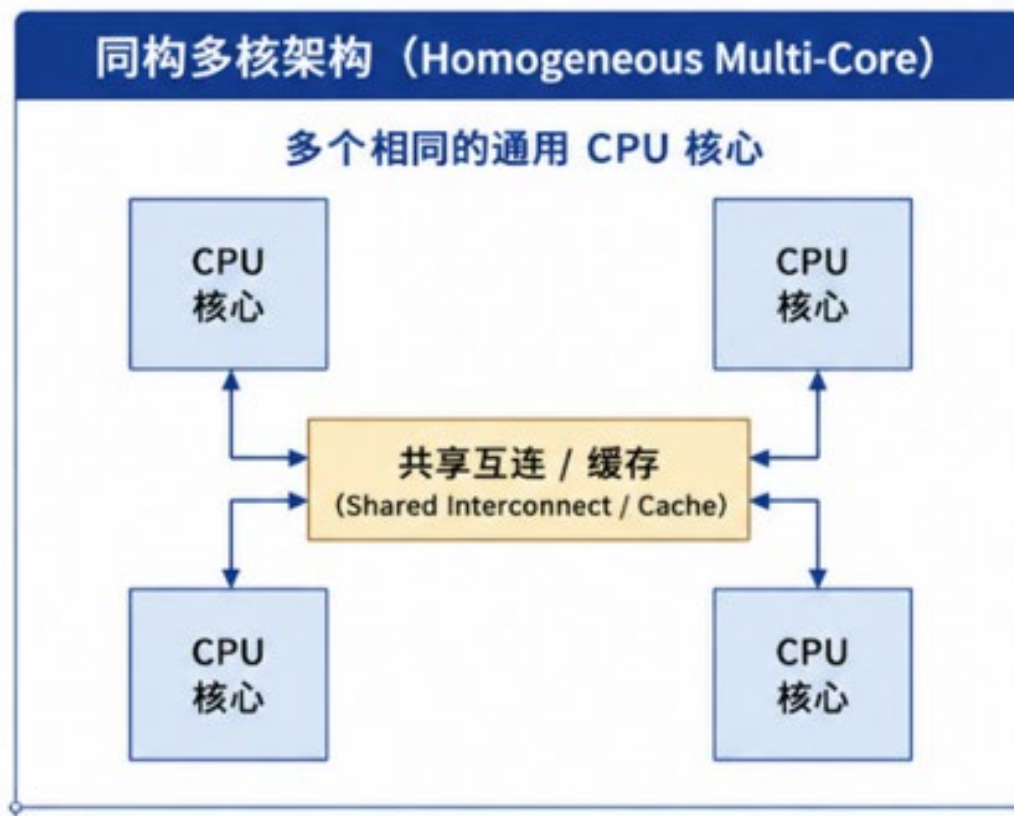
智能科学与技术系

核心问题：不同计算单元怎样在功耗预算内协同工作？



- | | | |
|----|--------------|-----------------------------|
| 01 | 异构SoC与计算单元分工 | 讲义3.3.1 · P56 – P58 |
| 02 | 统一内存与异构任务调度 | 讲义3.3.2 / 3.3.3 · P58 – P62 |
| 03 | 低功耗技术与专用架构 | 讲义3.4.1 / 3.4.2 · P62 – P67 |
| 04 | 协处理器与系统能效协同 | 讲义3.4.3 · P68 – P70 |

- 同构多核：集成多个相同的通用计算核心
- 异构计算：将不同架构、不同指令集、为不同任务优化的专用处理单元集成到同一芯片上，形成一个完整的片上系统，以期在能效和性能上达到新的极致

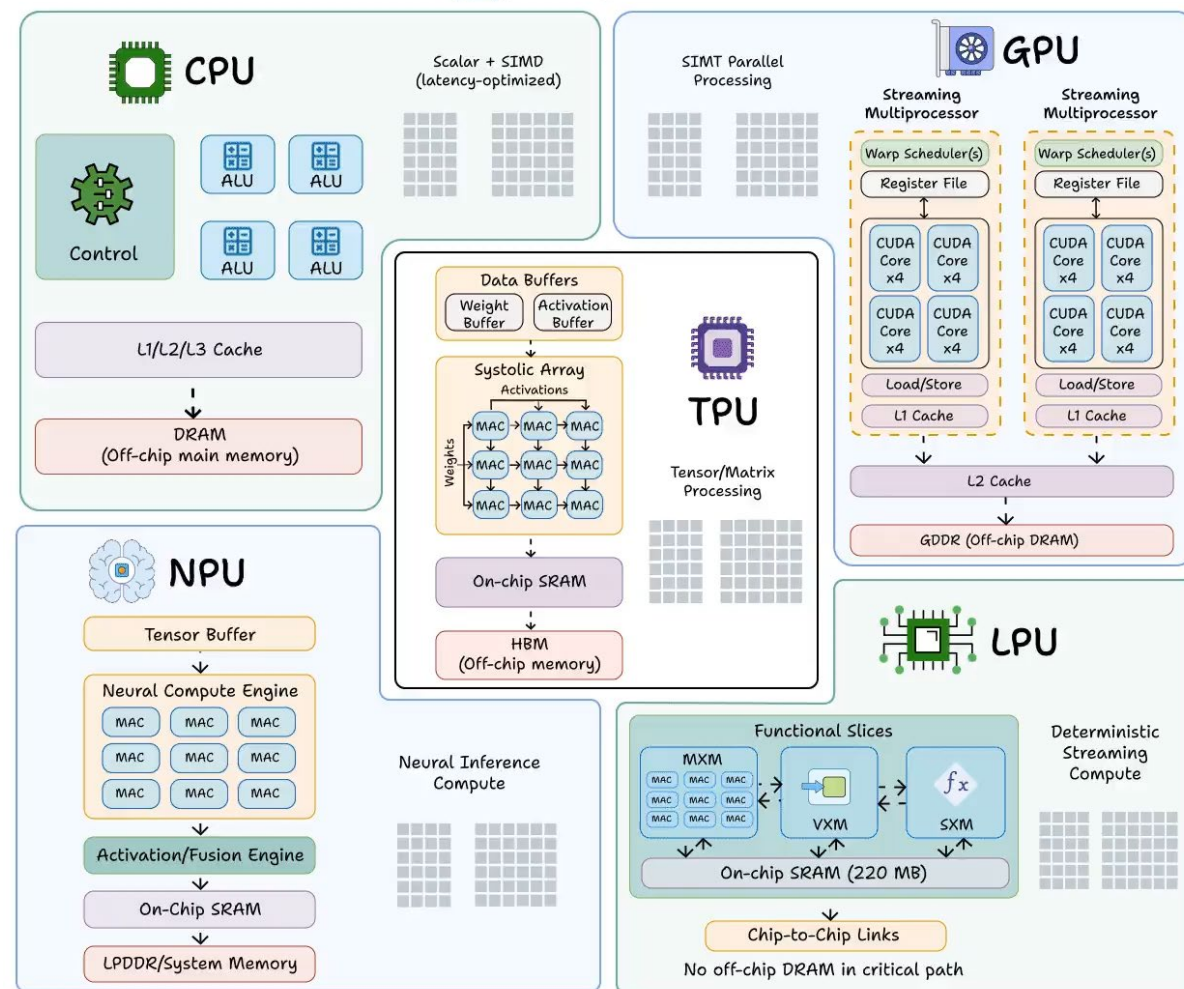


- **CPU：中央处理器**，控制核心处理复杂的控制逻辑、分支预测和任务调度
- **GPU：图形处理器**，规整的大规模并行运算，架构包含成千上万个简单核心
- **NPU：神经网络处理器**，受支持的神经网络算子，直接映射NN中的张量运算
- **ASIC：数字信号处理器及其他专用集成电路**，处理特定的、重复性高的信号处理或编码任务，如音频、视频编解码

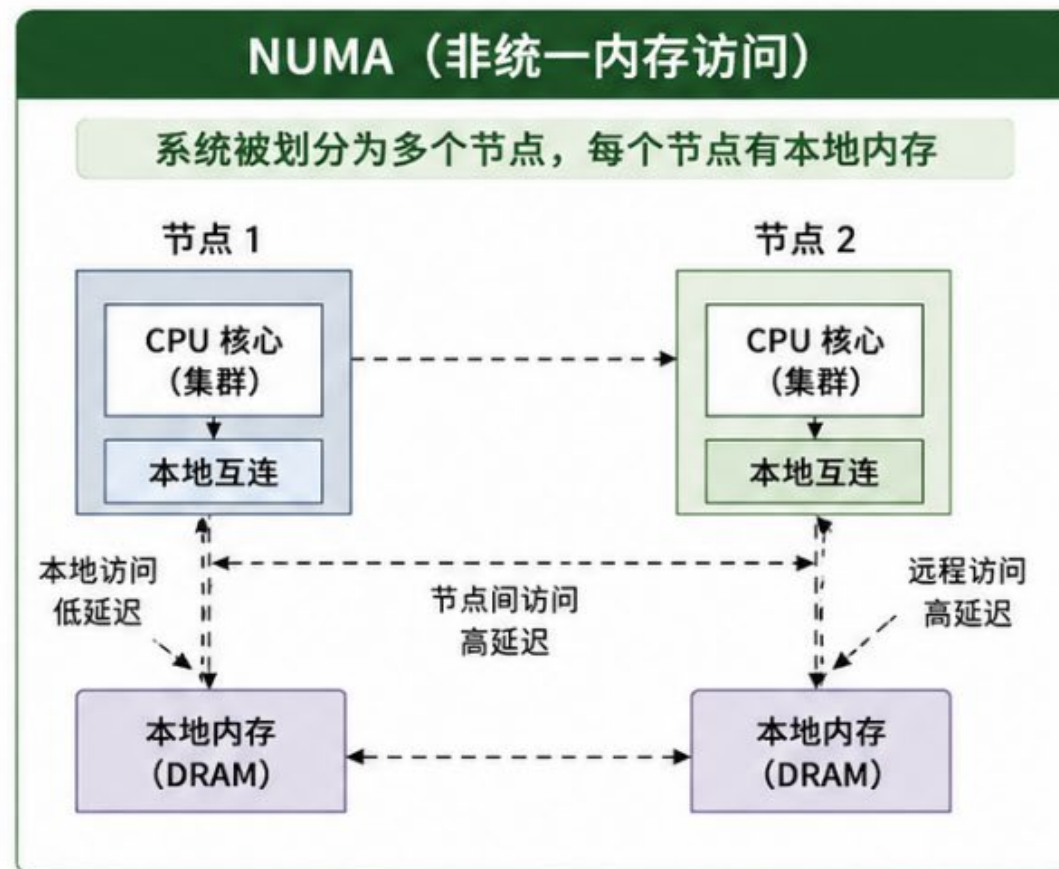
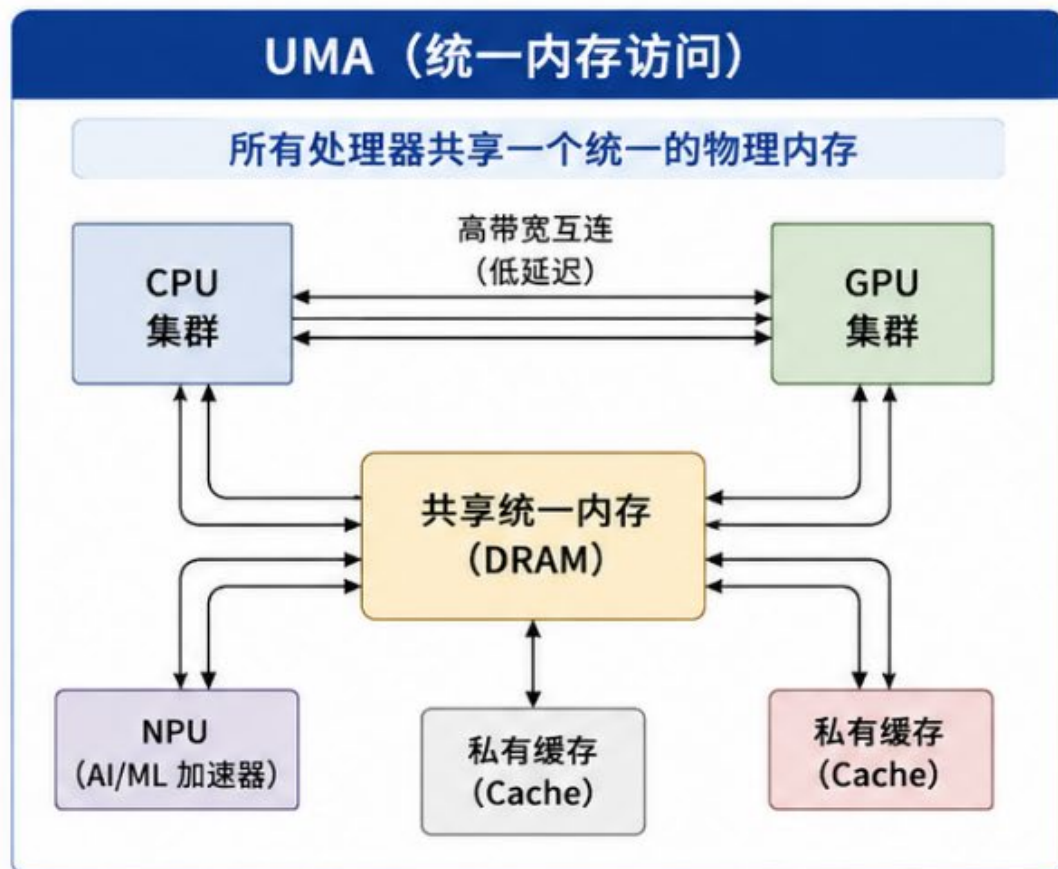
5 AI Compute Architectures

CPU vs GPU vs TPU vs NPU vs LPU

join.dailydoseofds.com

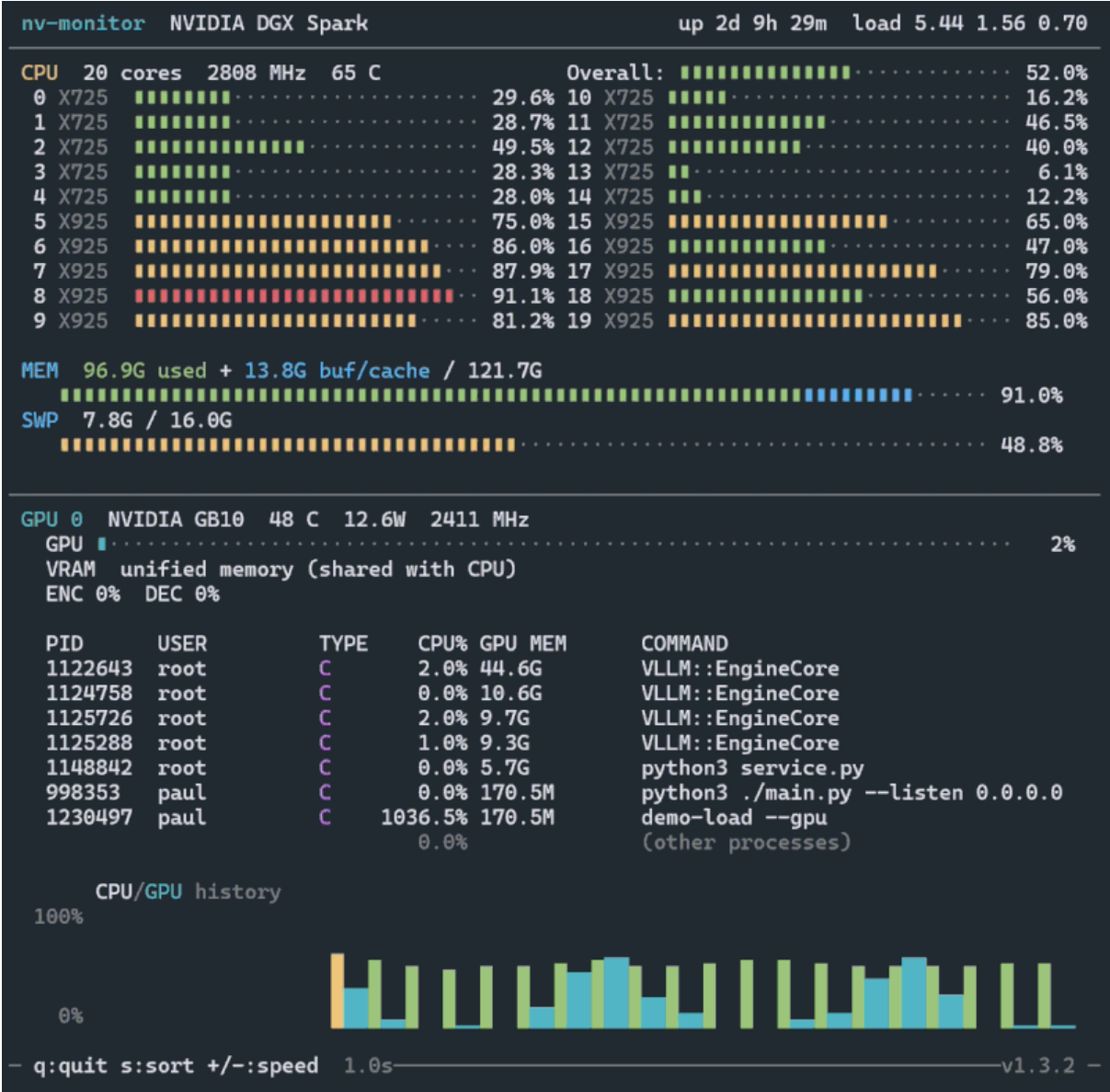
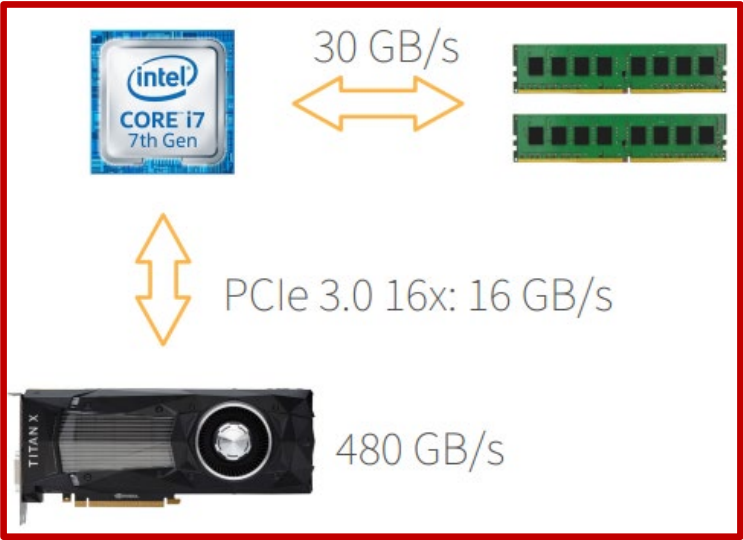


□ 系统中所有处理器（CPU、GPU、NPU）共享同一个统一的物理内存地址空间，任何处理器都可以直接访问同一份数据，无需在不同内存池之间进行显式拷贝



```
device = torch.device(
    'cuda' if torch.cuda.is_available() else 'cpu'
)
model = models.resnet50(pretrained=True)
model.load_state_dict(
    torch.load(f"model_state_dict.pth", map_location="CPU")
)
model.eval()

# Run a dummy inference
input_data = torch.rand(1, 3, 224, 224)
output_data = model(input_data)
```

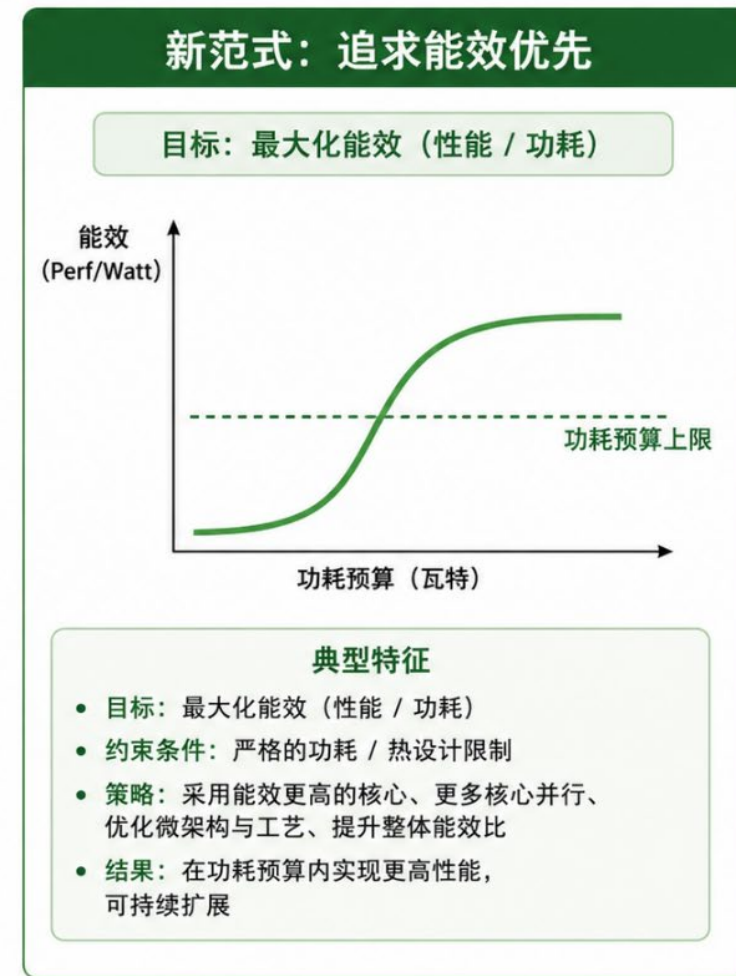
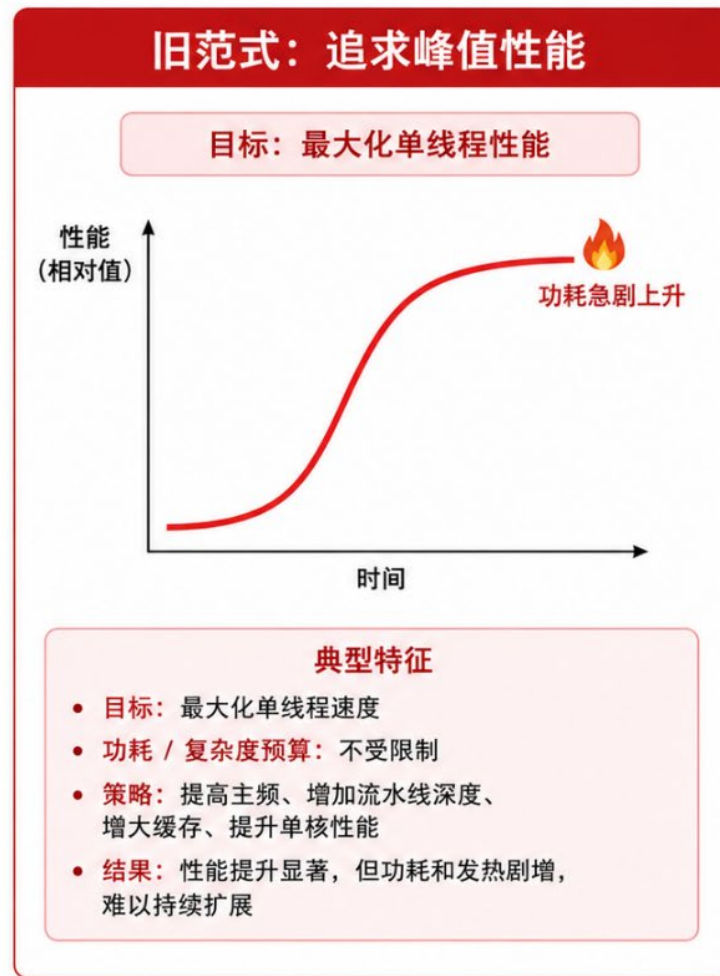


- 不同处理单元能力的不对称性要求调度器必须理解每个任务的计算特征
- 数据传输开销，数据传输的延迟和能耗可能超过计算本身
- 静态调度与编译时优化；动态运行时调度机制；能效导向的调度策略

特性	静态分配策略 (Static Allocation)	动态分配策略 (Dynamic Allocation)
核心思想	在任务开始执行前就确定好它将在哪个核心上运行，运行期间通常不迁移	在任务整个生命周期内，操作系统可以随时将其在不同核心之间进行迁移
工作流程	由程序员或系统配置预先设定任务的 CPU亲和性 （CPU Affinity），将其绑定到指定核心	调度器根据当前的系统负载、缓存热度等因素， 动态地 决定任务在何处执行。全局队列和局部队列都属于动态分配
优点	开销极小：无需运行时调度决策，无任务迁移开销 确定性高：行为可预测，适用于实时系统 避免迁移成本：保证了极强的缓存亲和性	灵活性强：能很好地适应负载波动 负载均衡：能将任务从繁忙核心转移到空闲核心，提高整体资源利用率
缺点	负载可能严重不均：如果被绑定的核心很忙，任务也必须等待，而其他核心可能空闲 不灵活：无法适应负载变化	引入开销：任务迁移会带来性能成本（缓存失效、TLB失效等） 调度决策本身有开销
适用场景	高性能计算：程序员明确知道如何分配任务以达到最优 实时系统：需要确保关键任务独占核心，避免调度不确定性 NUMA系统优化：避免远程内存访问	通用计算环境：几乎所有消费级和服务端操作系统都采用动态分配，以应对不可预测的、多样化的 workloads

- ❑ **“功耗墙”** 的硬约束
- ❑ **微架构**：从挖掘ILP转向挖掘线程级并行和数据级并行
- ❑ **系统架构**：异构计算
- ❑ **算法与软件**：能效感知

细粒度的ILP虽然能提升性能，但硬件开销带来的功耗代价超过性能收益；粗粒度的线程级并行和数据级并行在提供相当性能的同时，通常具备更好的能效特性



核心转变

追求“更快”
(性能最大化)



追求“更聪明”
(能效最大化)

在功耗墙限制下，设计重点从“峰值性能”转向“能效优化”。

1. 系统/运行时层 (软件/OS/运行时)

 粒度：毫秒 ~ 秒级
作用：动态调节系统
整体性能/功耗

代表技术：动态电压频率调节 (DVFS)



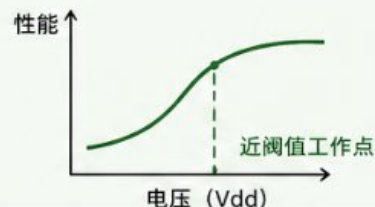
实现方式

- 运行时根据工作负载，动态调整 CPU/GPU 等的电压和频率
- 在性能和功耗之间取得平衡

2. 电路/逻辑层 (电路/逻辑设计)

 粒度：微秒 ~ 毫秒级
作用：优化电路工作点，
降低动态功耗

代表技术：近阈值计算 (Near-Threshold Computing)



- 在靠近器件阈值电压的工作点运行
- 在满足性能需求的前提下，显著降低电压，减少功耗
- 是 DVFS 的重要工作点选择

实现方式

- 设计电路在近阈值电压下稳定工作
- 优化电压 (Vdd)，获得更高能效
- 典型场景：移动设备、低功耗 SoC

3. 晶体管/物理层 (器件/物理实现)

 粒度：纳秒 ~ 微秒级
作用：降低待机功耗 (漏电)，
优化物理实现

代表技术：功率门控 (Power Gating)，漏电优化 (如堆叠-Vt)



- 功率门控：为空闲模块切断电源，消除漏电
- 漏电优化：采用高阈值器件、堆叠-Vt 等工艺技术，降低漏电流

实现方式

- 空闲模块通过电源开关断电
- 工艺层面采用低漏电器件结构
- 降低待机/静态功耗，提升能效



总结

低功耗技术需在不同层级协同优化：运行时动态调节 (DVFS) → 电路级优化工作点 (近阈值) → 物理级降低漏电 (功率门控/漏电优化)，实现系统整体能效最大化。

谢谢

Thank you

